

Ros2 Day3 hw2 AI 사용 보고서

휴머노이드 수습단원 이주원

목차

1. ai사용 부분
2. 대화내용
3. 깨달은 점

1. ai 사용 부분

이 과제에서 ai가 사용된 부분은 첫 번째로 차량의 위치를 갱신하는 방법이다. 차량이 움직이는 것을 어떻게 구현해야 할지 감이 안 잡혀서 물어보았다.

두 번째는 앞차와의 간격 처리다. 차들이 서로 겹치지 않게 해야 하고 뒤로 밀려서도 안 되는데, 이 둘을 어떻게 처리해야 하는지 확인을 요청했다.

세 번째는 폴더 구성이다. 노드 소스를 따로 담을 폴더를 만들어야 하는지 물어보았다.

네 번째는 타입 등록이다. 차량 위치를 배열로 넘기려 하니 신호가 화면 쪽에 도달하지 않아서 이유를 확인했다.

다섯 번째는 차량 생성이다. 일정 시간마다 차가 새로 나타나도록 하려면 어떻게 해야 하는지 물어보았다.

2. 대화 내용

차량을 움직이게 하려면 위치를 어떻게 갱신해야해?

차량 위치 갱신 로직은 vehicle_node에서 타이머 콜백으로 주기적으로 처리하는 게 핵심입니다. 흐름은 이렇습니다.

기본 구조
상태 변수 (멤버 변수로 유지)

```
cpp
float x_ = 0.0f;
float y_ = 0.0f;
float speed_ = 0.0f; // 현재 속도 (m/s)
float max_speed_ = 5.0f; // 파라미터로 뱌 값
float accel_ = 2.0f; // 가속도
float decel_ = 4.0f; // 감속도 (정지선 접근 시)
std::string light_state_ = "RED";
rcpp::Time last_time_;
```

타이머로 주기 갱신 (예: 50ms = 20Hz)

↓

위치 갱신은 차량 노드에서 타이머 콜백으로 주기적으로 처리하는 것이 핵심이라는 답을 받았다. 위치와 속도를 멤버 변수로 들고 있으면서 타이머가 돌 때마다 갱신하는 구조이고, 속도와 가속도 같은 값은 파라미터로 빼두는 것이 좋다는 설명을 들었다. 예시로는 50밀리초 주기가 제시되었다.

앞차와 간격 유지하는 부분과 뒤로 안 가게 막는 부분을 어떻게 해야 할까?

4초 동안 생각함 Designing a car-following system with speed safeguards.

차량이 여러 대라면 vehicle_node 내부에서 차량들을 배열/벡터로 관리하고, 각 차량이 "내 앞차"를 참조하는 구조로 가면 됩니다.

1. 차량 구조체로 관리

```
cpp
struct Vehicle {
    float x = 0.0f;
    float y = 0.0f;
    float speed = 0.0f;
    int id;
};

std::vector<Vehicle> vehicles_;
```

↓

앞차 간격은 차량들을 벡터로 관리하고 각 차량이 자기 앞차를 참조하는 구조로 가면 된다는 답을 받았다. 차량을 구조체로 만들어 위치와 속도를 함께 담는 방식을 안내받았는데, 나는 화면에 표시할 위치만 있으면 되어서 값 하나만 벡터에 담는 쪽으로 단순하게 구현했다.

nodes 폴더를 따로 만들어야해?

필수는 아니지만, 노드 3개짜리 패키지라면 관례적으로 나누는 게 유지보수에 좋습니다. ROS2 C++ 패키지에서 흔히 쓰는 구조 두 가지를 보여드릴게요.

옵션 1: 단일 패키지 + src 폴더 안에서만 분리 (가장 흔함)

노드가 3개뿐이고 서로 밀접하게 연관된 시뮬레이터라면, 하나의 패키지 안에 src/만 있어도 충분합니다.

nodes/ 폴더는 필수가 아니에요.

```
traffic_sim/
├── CMakeLists.txt
├── package.xml
├── include/
│   └── traffic_sim/
│       ├── traffic_light_node.hpp
│       ├── vehicle_node.hpp
│       └── gui_node.hpp
└── src/
```



폴더는 필수가 아니라는 답을 받았다. 노드가 세 개 정도면 하나의 패키지 안에서 소스 폴더만으로도 충분하고, 나누는 것은 유지보수를 위한 관례에 가깝다고 한다. 다만 내 경우에는 소스 폴더를 통째로 긁어와 Qt 실행 파일을 만드는 구조라 메인 함수가 겹치는 문제가 있어서 결국 폴더를 나누었다.

qRegisterMetaType 이 왜 필요해?

Qt의 시그널-슬롯이 스레드를 넘나들 때 문제가 되는 지점 때문입니다. ROS2 GUI 노드에서 특히 자주 마주치는 이슈예요.

왜 필요한가

ROS2 GUI 노드 구조를 보면 보통:

```
cpp
rclcpp::spin(node); // 이 스레드에서 구독 콜백 실행
```

이 `spin()` 은 보통 별도 스레드(QThread)에서 돌리고, Qt의 메인 이벤트 루프는 GUI 스레드에서 돌립니다. 즉 구독 콜백(ROS 스레드)에서 받은 데이터를 GUI 위젯(메인 스레드)에 반영해야 하는데, Qt 위젯은 자기 스레드가 아닌 곳에서 직접 건드리면 안 됩니다.

```
cpp
// ROS 콜백 스레드에서 이렇게 직접 하면 위험
```



타입 등록은 Qt의 신호가 스레드를 넘나들 때 문제가 되는 지점이기 때문이라는 답을 받았다. 노드를 돌려주는 부분은 별도 스레드에서 돌고 화면 쪽은 메인 스레드에서 도는데, 구독 콜백에서 받은 데이터를 화면에 반영하려면 스레드를 넘겨야 하고 화면에 있는 것은 자기 스레드가 아닌 곳에서 직접 건드리면 안 된다는 설명을 들었다. ROS와 Qt를 같이 쓰는 구조에서 자주 마주치는 문제라고 한다.

어떻게 해야 차량을 스폰할 수 있게 할까?

1초 동안 생각함 Designing vehicle spawning logic for a simulation.

차량 스폰은 크게 (1) 언제 스폰할지 트리거, (2) 스폰 가능한지 조건 체크, (3) vehicles_ 벡터에 추가, 이 세 부분으로 나눠서 설계하면 됩니다.

1. 스폰 트리거 방식 선택

A. 타이머 기반 자동 스폰 (가장 간단, 시뮬레이터에 적합)

```
cpp
// 파라미터로 스폰 주기 설정
double spawn_interval_ = 3.0; // 초
rclcpp::TimerBase::SharedPtr spawn_timer_;

spawn_timer_ = this->create_wall_timer(
    std::chrono::duration<double>(spawn_in ↓ al_),
    std::bind(&VehicleNode::trySpawnVehicle, this));
```

차량 생성은 세 부분으로 나누어 설계하면 된다는 답을 받았다. 언제 생성할지 정하는 부분, 생성이 가능한 상황인지 확인하는 부분, 그리고 벡터에 추가하는 부분이다. 생성 시점을 정하는 방법으로는 타이머를 쓰는 방식이 가장 간단하고 시뮬레이터에 적합하다는 설명을 들었다.

3. 깨달은 점

움직임을 구현할 때 위치를 직접 지정하는 것이 아니라 타이머가 돌 때마다 조금씩 갱신하는 방식이라는 것을 알게 되었다. 속도나 주기 같은 값을 파라미터로 빼두면 코드를 고치지 않고도 동작을 바꿀 수 있다는 점도 이번 과제에서 실제로 써보며 확인했다.

답변을 그대로 따라가지 않아도 된다는 것을 알게 되었다. 차량을 구조체로 만들라는 안내를 받았지만 내 경우에는 화면에 찍을 위치만 있으면 충분해서 값 하나만 담는 쪽으로 줄였다. 필요한 것이 무엇인지 먼저 따져봐야 한다는 점을 느꼈다.

폴더 구성에 정해진 답이 있는 것은 아니라는 것을 알게 되었다. 다만 내 빌드 설정에서는 나눌 수밖에 없었는데, 일반적인 권장 방식과 내 상황에서 필요한 것이 다를 수 있다는 점을 알았다.

ROS와 Qt를 같이 쓸 때 스레드 문제가 항상 따라온다는 것을 알게 되었다. hw2, hw3에 이어 이번에도 같은 지점이었는데, 이번에는 값이 배열이라 타입까지 등록해줘야 했다. 기능을 통째로 생각하지 않고 단계로 나누어 보는 것이 도움이 된다는 것을 알게 되었다. 차량 생성도 언제, 조건, 추가 세 부분으로 나누어 보니 각각은 간단한 코드였다.